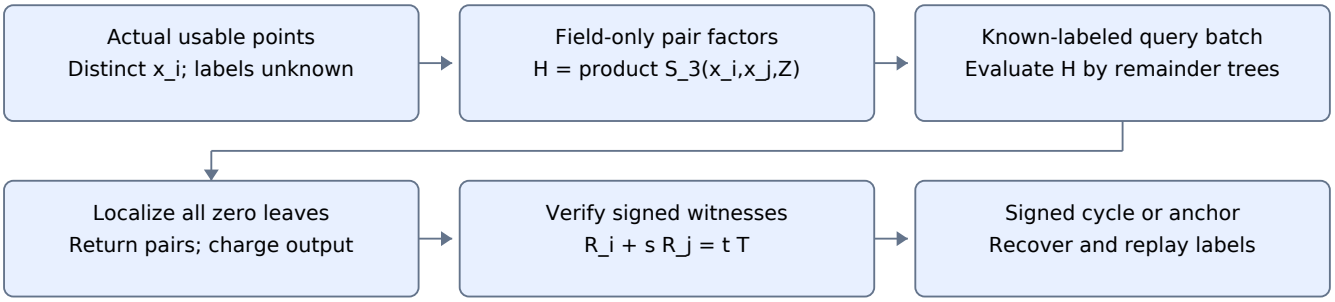
A coordinate polynomial index for known-label construction

Coordinate polynomial index and signed labels



Explicit degree n(n-1); quadratic information is retained.
An unbalanced even cycle can recover labels; balanced cycles need an anchor.
Complete cost improvement UNKNOWN. No benchmark or attack-speedup claim.

2026-10-29. **Implemented correctness prototype and derived cost model; no performance benchmark, attack improvement or hypothesis promotion.** The requested efficient construction into a small usable coordinate domain remains unestablished. This note supplies a concrete next constructor component rather than treating publication or a repeated obstruction as the result.

It extends the label-graph proposal with a coordinate-only, batched witness index and signed graph propagation. The original 34-file archive and its evidence are unchanged. The canonical construction problem remains open. PROTOCOL.md specifies the comparison still owed.

1. Domain and exact pair polynomial

Let E/F_p be y^2=x^3+A*x+b, p>3, with a prime-order subgroup G=<P> of order r>3 and Q in G. Enumerate and verify n actual subgroup points R_i, one chosen orientation per distinct usable abscissa x_i in the chosen domain. Their individual labels are initially unknown. The ambient domain may have B roots; n denotes usable coordinates after curve/subgroup filtering, not B.

For i<j define the ordinary third summation polynomial in Z:

```
F_ij(Z) = (x_i-x_j)^2*Z^2
        -2*((x_i+x_j)*(x_i*x_j+A)+2*b)*Z
        +(x_i*x_j-A)^2-4*b*(x_i+x_j).
H(Z) = product_(i<j) F_ij(Z).
D = deg(H) = n*(n-1).
```

With x_i != x_j, the addition formula gives exactly

```
F_ij(Z) = (x_i-x_j)^2
        * (Z-x(R_i+R_j)) * (Z-x(R_i-R_j)).
```

Both resulting points are finite and nonzero: equality up to sign would contradict distinct abscissae. Odd prime subgroup order also excludes nonzero two-torsion. Consequently H(x(T))=0 for finite T in G precisely when T has a signed pair witness using two different vertices. No self-pair witnesses are included. These assumptions are required, not certified by the polynomial API.

Build H and its factor product tree using x_i only. The leaves need constant field arithmetic and no inversion; constructing this index calls no curve addition. The high degree is explicit: this is a degree-D index of pair sums, not a low-degree vanishing polynomial for n coordinates and not a subquadratic representation of all pair information.

The ambient equation, such as X^B=c, remains available alongside curve, subgroup and exact chosen-base checks. Filtering does not preserve its exact two-term vanishing polynomial. The prototype

accepts any supplied usable set; it does not assert that an arbitrary known-label base has a small product set.

2. Batched tests and witness localization

Generate a batch of N known-labeled T_q=[a_q]P+[b_q]Q and obtain their actual point representations. Handle identity targets as independently verified early DLP opportunities when b_q is invertible. Use monic product/remainder trees to evaluate H at all finite x(T_q), including repeated coordinates.

Propagate only zero evaluations down the pair-factor tree. At each visited node, batch-evaluate its children at the active target coordinates; discard nonzeros. A leaf gives an unordered pair (i,j). Compute its actual signed point sums and verify the unique equation

$$R_i + s*R_j = t*T_q, \qquad s,t \text{ in } \{+1,-1\}.$$

Insert the edge label t*(a_q,b_q). Distinct x_i and odd subgroup order exclude ambiguity between the two leaf roots. Every matching pair is returned; many pairs per target are possible and their output/verification work is charged. No factor-root extraction, uncharged pair scan per hit or discrete-log table is used by pair_index.py.

With fast univariate polynomial arithmetic, let C be the number of returned pair/target incidences. Building the tree and testing/localizing one batch costs O~(n^2+N+C) field operations and O~(n^2+N+C) storage. Indeed the total polynomial degree at any level is D. After the first evaluation, the active query/node incidences at a level are at most a constant times the final leaf incidences. Summing over logarithmically many levels gives the stated soft bound. Repeated batches pay their own evaluation costs; precomputation is not silently amortized away. Point creation, filtering and scalar replay add costs.

The index avoids the quadratic curve additions used to build an explicit signed-pair dictionary, replacing them with quadratic polynomial work. This does not establish a lower complete time: polynomial products, reductions, data movement and allocation may be more expensive than a tuned pair table. Memory and elapsed time must be measured. Generic pair information is still materialized; this belongs with generic decomposition controls, not a claimed new algebraic attack exponent.

3. Exact coverage without extracting the pair-sum roots

When D<p, the squarefree part H/gcd(H,H') has degree K equal to the number of distinct target abscissae. All roots are rational and come from nonzero G points by the factor identity. A uniform target T in G therefore satisfies

$$\Pr(T \text{ finite and } H(x(T))=0) = 2*K/r.$$

This includes sign folding; it is not the probability that a graph becomes labelable. The identity event has probability 1/r and is recorded separately. The degree restriction avoids the inseparability case in the simple derivative formula; the prototype refuses that shortcut for D>=p. Coverage, graph connectivity and independent cycle defects are different quantities. No random graph or independent-edge model is assumed for actual coset points.

4. Signed cycles extend the positive-edge graph

An edge R_i+s*R_j=T_e transports a root expression as

$$R_j = -s*R_i + s*T_e.$$

Along a path write R_i=c_i*R_root+C_i with c_i in {+1,-1}, and keep the two known coefficients of C_i. A closed walk gives

$$R_{root} = a*R_{root}+C, \qquad (1-a)*R_{root}=C.$$

If a=-1, division by two recovers an explicit label for the root and hence its component. If a=1, the cycle leaves the root free; a verified anchor can still determine it. The criterion is a nontrivial product of path multipliers, not odd graph length. Three sum edges and one difference edge in a square give

2*R_0 = T_01-T_12+T_23-T_30.

Thus an even cycle can recover labels without an anchor. An all-sum even cycle and an all-difference cycle remain free. signed_labels.py implements both directions of each edge with their correct, generally different offsets.

Inconsistent coefficient labels on a verified graph yield a zero relation alpha*P+beta*Q=O. If beta!=0 modulo r, recover d=-alpha/beta and independently verify Q=[d]P. Preserve and charge these earlier DLP opportunities; do not silently discard them, infer an unverified scalar, or count algebraic graph updates as the whole constructor.

This weighted propagation is standard linear algebra on a signed graph. Its role here is an implemented extension of the previous positive-edge reference, not a claim that signed graphs or the summation polynomial are novel.

5. What was checked and what remains

The ten deterministic controls in test_pair_index.py check the generic cleared addition identity symbolically; all pair factors; every finite point of a tiny 823-order curve against independent pair enumeration; repeated evaluation coordinates; actual signed witnesses and label replay; a filtered seven-root multiplicative coset; invalid inputs; balanced/unbalanced cycles; and a cycle-defect DLP certificate. The fixture is the registered E/F_827: y^2=x^3+x+15, P=(2,822), r=823, with an explicitly known secret used only by the fixture and independent replay. See CURVE_FIXTURE.json for existing EC1/UID and registry source bindings. The index and graph routines receive no hidden logarithms. This is a unit correctness fixture, not a new benchmark, curve campaign or cryptographic-scale result. See the versioned CHECKS receipts.

PairIndex.lean attempts seven faithful lemmas: affine composition; the folded path invariant; the closed-walk constraint; root recovery; transport to the actual two-generator label expression; the signed-square example; and the generic cleared addition identity. It is registered by exact source hash with the pinned Lean v4.35.0-rc4 / Mathlib 021c46bdd042f67b538b9f9871d556a93713a9a9 native compiler and exhaustive module auditor, requiring seven audited theorems and zero failing/trusted exceptions. Unknown paths and altered bytes still fail. Local Lean/Lake are unavailable: these new proofs are UNBUILT locally, pending real native CI. Python refusal controls simulate outputs and are not proofs.

All 30 current refusal controls pass. The first gate-control attempt caught an outdated helper signature in the new registration; that failed output is preserved in GATE_CONTROLS.json. The corrected implementation uses current main's pinned native checker, binds all seven full theorem names and checks workspace input bytes before and after compilation/auditing. Its successful control receipt is GATE_CONTROLS-attempt2.json. No existing admission or refusal check was removed or weakened.

The path lemmas are statements about a module over a field; their concrete ZMod-r elliptic-subgroup instantiation, executable tree correctness, coverage counting and complete cost analysis are not formalized. The prior four ring identities, their source binding and historical audit evidence are unchanged. No full elliptic theorem or native check of the new source is claimed here.

The original requested efficient constructor remains partial. The witness oracle and signed label core are implemented; a complete driver, qualified matched cost comparison and subsequent relation-solver benefit remain owed. A first implementation/measurement must compare against the explicit signed pair dictionary and tuned reusable multi-target BSGS, and must keep the same chosen-base membership. Passing these controls or merging this PR cannot establish the requested improvement.

6. Audit, sources and rejected repeats

The current atlas was matched at revision f598670fd53cfbf91e6cb880e4e13b829c154149: all 204 lead/known-result files and three matcher sources were Git-blob-hash verified. See ATLAS_INPUTS.json, ATLAS_DOWNLOAD_RECEIPT.json and ATLAS_MATCHES.json. Its existing pair-table/root-finding lead already records generic decomposition oracles and cost-calibration failures. This proposal does not reclassify that track or promote a hypothesis. Related literature and the atlas do not establish global novelty for this implementation.

- Petit, Kosters, Messeng, PKC 2016: prior prime-field factor-base and summation-polynomial framework. The coordinate restriction is retained rather than credited as new. - Square-root Velu, Bernstein et al.: established index systems and fast polynomial tools. The original archive already implements paired progression certificates and index recovery; rediscovering that method was rejected as a new lead. Full certificate reuse also leads back to the optimized multi-target BSGS control. - Original block construction evidence: preserve its partial-cost erratum; no old timing is treated as a complete cost.

No new curve, isogeny, map or canonical isogeny-graph fact is asserted. The volcano-IC graph family and curve-catalog roles were checked for applicability; their vertices/edges remain current. The new diagram is algorithm flow, rendered from the editable diagram source, and contains no performance data. No search or performance job is scheduled by this research note.